

WP1 — Chi tiết x86 vs ARM

WP1 chi tiết — Hai máy, hai thí nghiệm: x86_64 và ARM

Tài liệu cho thành viên nhóm, dùng KÈM bảng tra cứu WP1_bang_tra_cuu.md (bảng = tra nhanh 12 file; file này = hiểu vì sao và vận hành chi tiết từng máy).

0. Vì sao phải hai máy — và nguyên tắc vàng

Đề bài đặt hai câu hỏi ở hai nơi khác nhau: **RQ1** cần baseline trên server x86_64 (PQC vs RSA/ECC tốn kém ra sao), **RQ2** cần máy ARM/SBC (tối ưu NEON có làm PQC khả thi không). Hai máy không phải “chạy lại cho đủ” mà là **hai thí nghiệm độc lập**.

Ba nguyên tắc vàng xuyên suốt:

1. **Binary là tài sản theo máy.** Mọi thứ trong ~/pqc build trên máy nào chỉ chạy trên máy đó (bản opt còn gắn chặt với đúng con CPU build ra nó — DIST_BUILD=OFF nghĩa là “for use on a single machine” theo CONFIGURE.md liboqs). Không copy binary chéo máy.
2. **Số đo là tài sản theo máy.** Chỉ so trực tiếp hai số đo *trên cùng một máy*. Muốn “bắc cầu” giữa hai máy: so **tỉ lệ** (speedup = opt/ref của từng máy), không so số thô (xem mục 4).
3. **Một bộ script cho cả hai.** Không có “bản script ARM” riêng — cùng repo, cùng lệnh; khác biệt nằm ở chỗ script *tự thích nghi* (khối uname -m trong build_liboqs.sh, linux-tools-\$(uname -r) trong install_prereqs.sh) và ở *kịch bản vận hành* dưới đây.

PHẦN A — Trên PC x86_64 (máy LOQ)

Vai trò: baseline RQ1 + nơi phát triển/gỡ lỗi toàn bộ + nơi duy nhất chạy cụm bằng-chứng-\$7.3 (cross-compile, Docker).

A1. Chuỗi lệnh và KỲ VỌNG THẤY GÌ ở mỗi bước

```
cd <gốc repo> && chmod +x scripts/*.sh
bash scripts/install_prereqs.sh
```

Kỳ vọng: apt chạy ba đợt; hộp thoại tshark hiện ra → chọn **Yes**. Kiểm: gcc --version (≈13.x), /usr/bin/time --version.

```
bash scripts/build_openssl.sh
```

Kỳ vọng: clone đúng tag 3.6.2 (~1 phút), Configure, make ~15-30 phút, make test thêm khá lâu (chạy đủ MỘT lần để lấy bằng chứng; các lần sau nếu rebuild dùng SKIP_TESTS=1). Kết thúc phải thấy openssl version in 3.6.2 và dòng DONE. Next step: source scripts/setenv.sh. Sinh ra: ~/pqc/openssl/, docs/openssl.commit.

```
source scripts/setenv.sh
bash scripts/verify_env.sh
```

Kỳ vọng: Activated OpenSSL from: ... + PASS. Report written to docs/env_report_x86_64.txt. Mở report thấy gcc/cmake/perl, cờ RELEASE từ Makefile, governor, số thuật toán ML-KEM/ML-DSA.

make

Kỳ vọng: build/bench_evp xuất hiện — harness WP2 sẵn sàng.

```
bash scripts/build_liboqs.sh ref # nghiệm thu chuỗi liboqs
bash scripts/build_liboqs.sh opt # TÙY CHỌN: dữ liệu phụ "hiệu ứng AVX2"
```

Kỳ vọng đọc trong header của speed_kem (chạy ~/pqc/src/liboqs/build-ref/tests/speed_kem ML-KEM-768): - Dòng OpenSSL enabled: Yes (OpenSSL 3.6.2 ...) → liboqs link ĐÚNG OpenSSL tự build (chống crypto-failure đạt). - Cây **ref**: CPU exts compile-time: SSE SSE2 (baseline x86-64 — vẫn là C “thuần”), AES: OpenSSL, OQS_OPT_TARGET=generic. - Cây **opt**: thêm -march=native, CPU exts: ADX AES AVX AVX2 BMI1 BMI2 ..., AES: NI, OQS_OPT_TARGET=auto.

Số mẫu đã đo thật trên LOQ (10/6/2026), ML-KEM-768 — để đồng đội biết “trông như thế nào là đúng”:

Op	ref (μ s)	opt (μ s)	Speedup	Kiểm chéo bằng cycles
keygen	30.985	10.705	$\times 2.89$	83190/28723 = $\times 2.90$ ✓
encaps	30.963	11.255	$\times 2.75$	✓
decaps	36.250	13.718	$\times 2.64$	✓

Đọc số này cho đúng: $\times 2.6$ – 2.9 là “hiệu ứng tối ưu x86 trọn gói” (AVX2 đóng vai chính, kèm AES-NI...), KHÔNG viết “AVX2 nhanh $\times 2.9$ ”. Hai thước đo độc lập (μ s và cycles) cho cùng tỉ lệ = phép đo lành mạnh.

```
bash scripts/fetch_pqclean.sh clean # 6 file .a tách riêng thuật toán (V)
bash scripts/fetch_pqclean.sh avx2 # tùy chọn: so code-size clean vs avx2
```

Kiểm: size ~/pqc/src/PQClean/crypto_kem/ml-kem-768/{clean,avx2}/lib*.a — avx2 thường TO hơn clean (cái giá bộ nhớ của tốc độ — nhận định Analysis đẹp).

A2. Cụm bằng-chứng §7.3 — CHỈ chạy trên x86

```
sudo apt install gcc-aarch64-linux-gnu g++-aarch64-linux-gnu # một lần
bash scripts/cross_build_liboqs.sh # binary ARM generic -> ~/pqc/liboqs
docker build -f docker/Dockerfile.x86_64 -t nt219-pqc . # cần Docker
bash scripts/docker_buildx.sh # tùy chọn: image đa kiến trúc
```

Vai trò của cả ba: **bằng chứng tái lập** cho báo cáo (mục Build & Reproducibility). Binary cross KHÔNG dùng để đo.

A3. Trước các phiên đo “lấy số thật” (WP2)

```
sudo cpupower frequency-set -g performance # cố định governor (§7.4)
```

Laptop để governor mặc định cho stdev to (đã thấy: keygen ref stdev 11.4μ s/mean 31μ s). Đo xong có thể trả về powersave.

A4. Dữ liệu máy này sinh ra (commit vào repo)

data/raw/x86_64/* .csv|txt, docs/env_report_x86_64.txt, docs/{openssl,liboqs,pqc}

PHẦN B — Trên máy ARM

Vai trò: trả lời RQ2. Đây KHÔNG phải “chạy lại phần A trên máy khác” — trọng tâm dịch chuyển: bench_evp/OpenSSL vẫn chạy để có baseline ARM, nhưng **ngôi sao là cặp liboqs ref/opt** (lần đầu tiên công tắc NEON thật sự có nghĩa).

B0. Chọn máy — và đặc tính ảnh hưởng trực tiếp tới thí nghiệm

Máy	CPU	NEON	ARM Crypto Ext (AES/SHA2)	PMU đếm chu kỳ	Lưu ý
Pi 4 hosted (vd Mythic Beasts)	Cortex-A72 (BCM2711)	Có	KHÔNG (Broadcom không mua license)	Bare-metal → có thể nạp module pqax (cần sudo + linux headers)	Máy IPv6-only: GitHub chưa có IPv6 — outbound đi qua NAT64/DNS64 của nhà cung cấp; nếu git clone treo, kiểm tra cấu hình DNS64 theo docs của họ
Oracle Ampere A1	Neoverse-N1	Có	Có	VM: PMU thường KHÔNG expose cho guest → wall-clock	Free tier; chọn image Ubuntu , tránh Oracle Linux (RPM cài lib vào lib64 → lệch đường dẫn script)
AWS t4g	Graviton2	Có	Có	Như trên	Free tier tới 31/12/2026

Điểm phương pháp luận hay (ghi vào báo cáo): trên Pi 4, vì chip KHÔNG có ARM Crypto Extensions, các cờ ARM_AES/SHA2 tự-dò sẽ OFF ở CẢ HAI cây — cặp ref/opt càng “sạch”, delta đúng bằng NEON. Trên Ampere/Graviton, AES/SHA2 bật Ở CẢ HAI cây như nhau → delta vẫn chỉ là NEON. Cả hai trường hợp đều là paired comparison hợp lệ (\$7.5), chỉ khác baseline.

B1. Chuỗi lệnh đầy đủ + thời gian dự kiến

```
git clone <repo> && cd <repo> && chmod +x scripts/*.sh
bash scripts/install_prereqs.sh # dòng linux-tools-$(uname -r) phát
SKIP_TESTS=1 bash scripts/build_openssl.sh # Pi 4: make đã ~30-60+ phút; t
source scripts/setenv.sh
bash scripts/verify_env.sh # -> docs/env_report_aarch64.txt
make # bench_evp bản ARM (baseline RQ1 ph
bash scripts/build_liboqs.sh ref # BẮT BUỘC
bash scripts/build_liboqs.sh opt # BẮT BUỘC
bash scripts/fetch_pqclean.sh clean
bash scripts/fetch_pqclean.sh aarch64 # trực đối chứng NEON thứ hai
```

(Một lần trước khi rời máy: chạy `bash scripts/build_openssl.sh` với `FORCE=1` KHÔNG `SKIP_TESTS` qua đêm để có bằng chứng `make test pass` trên ARM — ghi vào report.)

B2. Kỳ vọng thấy gì ở cặp liboqs — KHÁC với x86 thế nào

Chạy `speed_kem` ML-KEM-768 trên từng cây: - Target platform: `aarch64-Linux-...` (không còn `x86_64`). - Cây **ref**: build log cmake phải có `-DOQS_USE_ARM_NEON_INSTRUCTIONS=OFF` (khỏi `uname -m` của script lần đầu thức dậy — trên x86 khối này im lặng). KHÔNG được thấy NEON trong CPU exts. - Cây **opt**: `NEON=ON + -mcpu=native`. Trên Pi 4 đừng chờ thấy AES/SHA2 (chip không có); trên Ampere thì có ở cả hai cây. - `opt` phải nhanh hơn `ref` rõ rệt → đó là “×B” của RQ2. Nếu hai cây ra số xấp xỉ: nghi build sai trước (đọc lại log cmake từng cây), nghi thuật toán không có đường NEON ở 0.14.0 sau (tra ALGORITHMS.md) — đừng vội kết luận “NEON vô dụng”.

Lưu ý generic đổi nghĩa theo máy: `x86` → `-march=x86-64`; `ARM64` → `-mcpu=cortex-a53` (CONFIGURE.md). Tức “ref ARM” và “ref x86” là hai baseline khác nhau — thêm một lý do cấm so chéo.

B3. PMU — đếm chu kỳ chính xác (tùy chọn, dễ bỏ qua)

ARM không có `rdtsc`; muốn cycles chính xác cần PMU + kernel module (CONFIGURE.md: thiếu module mà build `USE_ARM_PMU=1` → `speed_kem` chết “Illegal Instruction”). Thứ tự ĐÚNG: nạp module trước (`clone mupq/pqax` → `cd enable_ccr` → `make install`, cần `sudo` + `linux headers`) → RỒI MỚI `USE_ARM_PMU=1 FORCE=1 bash scripts/build_liboqs.sh opt`. Pi bare-metal: khả thi. VM cloud: PMU thường không expose → bỏ qua, dùng wall-clock (`CLOCK_MONOTONIC_RAW` trong `bench_evp` + cột Time của `speed_kem`) và ghi một dòng Limitations — để bài §13 cho phép.

B4. Governor trên ARM

```
sudo cpupower frequency-set -g performance
# nếu cpupower thiếu: echo performance | sudo tee /sys/devices/system/cpu/cpufreq/policy0/scaling_governor
```

Pi mặc định `ondemand` — dao động tần số làm nhiều số đo y như laptop.

B5. Dữ liệu máy này sinh ra (commit vào repo trước khi trả máy!)

`data/raw/aarch64/*`, `docs/env_report_aarch64.txt`. Máy thuê có thể bị thu hồi/xóa — **push ngay sau mỗi phiên đo**.

4. So sánh thế nào cho ĐÚNG — bài học bốn con số

Sau hai máy, riêng liboqs ML-KEM-768 bạn có bốn con số. Quy tắc:

Phép so	Hợp lệ?	Trả lời câu gì
<code>ref-x86 ↔ opt-x86</code> (cùng máy)	✓	Hiệu ứng tối ưu x86/AVX2 (phụ, RQ1) — đã đo: ×2.6–2.9
<code>ref-ARM ↔ opt-ARM</code> (cùng máy)	✓	Hiệu ứng NEON — chính là RQ2
<code>opt-x86 ↔ opt-ARM</code> (chéo máy)	× số thô	Đổi hai biến cùng lúc (CPU + loại tối ưu) — vô nghĩa
<code>(opt/ref)x86 ↔ (opt/ref)ARM</code> — tỉ lệ	✓	“Tối ưu vi kiến trúc đáng giá ×A trên server, ×B trên SBC” — câu kết luận đẹp nhất của giả thuyết đề bài

Cùng logic áp cho bench_evp (RQ1): PQC-vs-RSA so trong từng máy; chéo máy chỉ so *tỉ số PQC/RSA* (“trên ARM, khoảng cách PQC-RSA gần/co thể nào so với x86”).

5. Bẫy đã biết — phân theo máy

Bẫy	x86	ARM	Cách né
Quên source setenv.sh ở terminal mới → openssl 3.0 hệ thống CRLF/chmod khi file đi từ Windows	✓	✓	which openssl trước khi đo sed -i 's/\r\$//' scripts/*.sh; chmod +x scripts/*.sh
Governor mặc định → stdev to linux-tools-generic không khớp kernel cloud → perf chết	✓	✓	mục A3/B4
Image Oracle Linux (RPM) → lib64 lệch đường dẫn	—	✓	script đã cài thêm linux-tools-\$(uname -r)
IPv6-only (Pi hosted) → GitHub không IPv6	—	✓	luôn chọn image Ubuntu
Gõ fetch_pqclean.sh aarch64 trên PC (hoặc avx2 trên ARM)	✓	✓	NAT64/DNS64 của nhà cung cấp; clone treo thì kiểm DNS64 guard trong script chặn sớm — thấy ERROR là đúng
Build USE_ARM_PMU=1 khi chưa nạp module → Illegal Instruction	—	✓	mục B3: module trước, build sau; hoặc bỏ PMU
Quên push data trước khi trả máy thuê	—	✓	push sau MỖI phiên đo

6. Checklist bàn giao (tick từng máy)

x86_64: ☐ env_report_x86_64.txt PASS · ☐ make test OpenSSL pass (1 lần) · ☐ bench_evp chạy · ☐ liboqs ref (+opt) có số speed_kem lưu vào data/raw/x86_64 · ☐ PQCclean clean 6/6 .a · ☐ cross_build ra “ARM aarch64” · ☐ docker build pass · ☐ 4 file docs/*.commit có mặt

ARM: ☐ env_report_aarch64.txt PASS · ☐ liboqs ref VÀ opt, speed_kem hai cây lưu data/raw/aarch64 · ☐ opt nhanh hơn ref (ghi tỉ lệ) · ☐ PQCclean clean + aarch64 · ☐ make test OpenSSL pass (1 lần, qua đêm) · ☐ đã push toàn bộ data trước khi rời máy